Code

Hypergraphe

```py
from typing import List

class Arete:
    cardinal: int
    field: int
    indice: int

    """
    Arête d'un hypergraphe.
    - sommets : bitfield des éléments des sommets
    - cardinal : nombre d'éléments dans sommets
    """
    def __init__(self):
        self.cardinal = 0
        self.field = 0
        self.indice = -1

    @property
    def sommets(self):
        sommets = []
        field = self.field

        i = 0
        while field != 0:
            if field & 1 == 1:
                sommets.append(i)
            i += 1
            field >>= 1

        return sommets

    def copier(self):
        a = Arete()
        a.field = self.field
        a.cardinal = self.cardinal
        a.indice = self.indice
        return a

    def augmenter(self, *sommets: int):
        for s in sommets:
            if not self.contient(s):
                self.field |= 1 << s
                self.cardinal += 1

    def reduire(self, sommet: int):
        if self.contient(sommet):
            self.field &= ~(1 << sommet)
            self.cardinal -= 1

    def dans(self, a):
        return (self.field | a.field) == a.field

    def contient(self, sommet: int):
        return self.field & (1 << sommet) != 0

    def __repr__(self):
        return f"Arete(sommets={self.sommets}, indice={self.indice})"


class Hyper:
```

```
    aretes: List[Arete]
    ordre: int

    """
    Hypergraphe.
    - ordre : nombre de sommets (numérotés 0..ordre-1)
    - aretes : liste d'objets Arete
    """
    def __init__(self, ordre: int):
        self.ordre = int(ordre)
        self.aretes = []

    @property
    def nb_aretes(self):
        return len(self.aretes)

    def decompte(self, sommet: int):
        return sum((1 if a.contient(sommet) else 0) for a in self.aretes)

    def est_valide(self):
        """
        Vérifie que chaque arête a cardinal > 0 et que chaque sommet de 0..ordre-1
        apparaît dans au moins une arête.
        """
        if any(a.cardinal == 0 for a in self.aretes):
            return False

        couvert = 0
        for a in self.aretes:
            couvert |= a.field

        while couvert != 0:
            if couvert & 1 == 0:
                return False
            couvert = couvert >> 1

        return True

    def retirer_sommet(self, sommet: int):
        for a in self.aretes:
            a.reduire(sommet)

    def copier(self):
        h = Hyper(self.ordre)
        h.aretes = [a.copier() for a in self.aretes]
        return h

    def est_acyclique(self):
        h = self.copier()

        restant = set(range(h.ordre))
        sommets_compte = [h.decompte(s) for s in range(h.ordre)]

        retirer = []
        while True:
            sortir = True

            # On enlève tous les sommets dans moins d'une arête
            retirer.clear()
            for s in restant:
                if sommets_compte[s] <= 1:
                    retirer.append(s)
                    sortir = False
```

```
            for s in retirer:
                restant.discard(s)
                h.retirer_sommet(s)

            # On enlève toutes les arêtes inclues dans une autre
            for indice, a in enumerate(h.aretes):
                if a.cardinal > 0:
                    for b in h.aretes:
                        if a.dans(b) and a.indice != b.indice:
                            for s in a.sommets:
                                sommets_compte[s] -= 1
                            h.enlever_arete(indice)
                            sortir = False
                            break

            # On continue éventuellement l'algorithme
            if sortir or len(restant) == 0:
                break

        return len(restant) == 0

    # Renvoie l'arbre si la requête est acyclique, None sinon
    def arbre_jointure(self):
        arbre = [[] for _ in range(self.nb_aretes)]

        h = self.copier()

        restant = set(range(h.ordre))
        sommets_compte = [h.decompte(s) for s in range(h.ordre)]

        retirer = []
        while True:
            sortir = True

            # On enlève tous les sommets dans moins d'une arête
            retirer.clear()
            for s in restant:
                if sommets_compte[s] <= 1:
                    retirer.append(s)
                    sortir = False
            for s in retirer:
                restant.discard(s)
                h.retirer_sommet(s)

            # On enlève toutes les arêtes inclues dans une autre
            for indice, a in enumerate(h.aretes):
                if a.cardinal > 0:
                    for b in h.aretes:
                        if a.dans(b) and a.indice != b.indice:
                            arbre[a.indice].append(b.indice)
                            arbre[b.indice].append(a.indice)
                            for s in a.sommets:
                                sommets_compte[s] -= 1
                            h.enlever_arete(indice)
                            sortir = False
                            break

            # On continue éventuellement l'algorithme
            if sortir or len(restant) == 0:
                break

        return arbre if len(restant) == 0 else None

```

```
    def ajouter_arete(self, *aretes: Arete):
        for a in aretes:
            a.indice = len(self.aretes)
            self.aretes.append(a)

    def enlever_arete(self, indice: int):
        del self.aretes[indice]

    def __repr__(self):
        return f"Hyper(ordre={self.ordre}, nb_aretes={self.nb_aretes}, aretes={self.aretes})"
```

Table

```py
from csv import reader
from typing import Dict, List, Self, Set

class Table:
    name: str  # Nom de la table
    rows: List[tuple]  # Lignes

    columns: List[str]  # En-tête

    def __init__(self, name):
        self.name = name
        self.rows = []
        self.columns = []

    @property
    def len(self):
        return len(self.rows)

    # Mise à jour du nom de colonne
    def rename(self, index: int, nname: str):
        self.columns[index] = nname

    # Importation d'un fichier csv
    def import_from(self, path: str):
        self.rows = []

        with open(path, mode="r") as file:
            csv = reader(file)

            self.columns = csv.__next__()
            self.rows.extend([tuple(line) for line in csv])

    # Projection et sélection
    def project(self, selection: List[bool], validate):
        switch = []

        for i, b in enumerate(selection):
            if b:
                switch.append(i)

        dedup = set()
        for row in self.rows:
            if validate(row):
                dedup.add(tuple(row[i] for i in switch))

        self.columns = [self.columns[i] for i in switch]
        self.rows = list(dedup)

    def __repr__(self):
        return f"Table(labels={self.columns}, rows_count={len(self.rows)})"

```

```
    def print(self):
        print("Labels ", self.columns)
        print(" - " + "\n - ".join(str(r) for r in self.rows))
```

Requête

```py
from typing import List

class Atom:
    name: str                   # Nom de la table de l'atome
    index: int                  # Indice de l'atome
    variables: set[str]         # Variables présentes dans l'atome

    original: tuple[str]

    def __init__(self, name, *variables):
        self.name = name
        self.variables = set(variables)
        self.record = { v:i for i, v in enumerate(variables) }
        self.original = variables

        self.variables.discard('_')

    def __repr__(self):
        return f"{self.name}({ ', '.join(str(v) for v in self.original) })"

class Head:
    name: str               # Nom de la tête pour faire joli
    variables: tuple[str]   # Variables présentes dans la tête

    def __init__(self, name: str, *variables):
        self.name = name
        self.variables = variables

    def __repr__(self):
        return f"{self.name}({ ', '.join(str(v) for v in self.variables) })"

class Query:
    atoms: tuple[Atom]      # Atomes de la requête
    head: Head              # Tête de la requête

    variables: set[str]     # Variables présentes dans la requête

    def __init__(self, head: Head, *atoms):
        self.head = head
        self.atoms = atoms
        self.variables = set()

        for i, atom in enumerate(self.atoms):
            atom.index = i
            self.variables = self.variables.union(atom.variables)

    def __repr__(self):
        return f"{self.head} :- {', '.join(str(a) for a in self.atoms)}"
```

Instance

```py
import time
from collections import defaultdict
from csv import reader
from typing import Dict, List, Self, Set

from table import Table
from hyper import Arete, Hyper
from request import Atom, Head, Query
```

```python

class JoinNode:
    atom: Atom              # Atome du noeud
    table: Table            # Table correspondant à l'atome

    record: Dict[str, int]  # Annuaire de la table du noeud
    variables: Set[str]          # Variables pertinentes du noeud

    children: List[Self]    # Enfants dans l'arbre de jointure
    root: bool    # Parent dans l'arbre de jointure

    def __init__(self, atom: Atom, table: Table):
        self.atom = atom
        self.table = Table(table.name)

        self.record = dict()
        self.variables = set()

        self.children = []
        self.root = False

        self.table.columns = table.columns.copy()
        self.table.rows = table.rows

        for i, v in enumerate(atom.original):
            if v != '_':
                self.variables.add(v)
                self.record[v] = i
                self.table.columns[i] = v
            else:
                self.table.columns[i] = '_'

    # Pour ajouter un enfant
    def push_child(self, child: Self):
        self.children.append(child)

    # Semi-jointure
    def semi_join(self, node: Self):
        intersection = sorted(self.variables.intersection(node.variables))

        index = set(
            tuple(row[node.record[v]] for v in intersection) for row in node.table.rows
        )

        self.table.project(
            [v in self.variables for v in self.table.columns],
            lambda row: tuple(row[self.record[v]] for v in intersection) in index
        )

        # Mise à jour des colonnes (ne garder que les variables, pas les "_")
        self.record = {v: i for i, v in enumerate(self.table.columns) if v != '_'}

    def patch_rows(self, row_a, row_b, selection: list[tuple[bool, int]]):
        return tuple(row_a[i] if a else row_b[i] for a, i in selection)

    # Jointure complète
    def full_join(self, node: Self, head: Head, strict):
        intersection = sorted(self.variables.intersection(node.variables))
        index = defaultdict(list)

        # Construction de la sélection
        selection = []
        columns = []
```

```
72          vars_iter = head.variables if strict else sorted(self.variables.union(node.variables))
73          for v in vars_iter:
74              if v in self.variables or v in node.variables:
75                  current = v in self.variables
76                  columns.append(v)
77                  selection.append((current, self.record[v] if current else node.record[v]))
78              else:
79                  raise ValueError(f"Variable {v} non présente lors de la jointure.")
80
81          # Construction du résultat
82          for row in node.table.rows:
83              key = tuple(row[node.record[v]] for v in intersection)
84              index[key].append(row)
85
86          result = []
87          for row in self.table.rows:
88              key = tuple(row[self.record[v]] for v in intersection)
89
90              if key in index:
91                  for row2 in index[key]:
92                      result.append(self.patch_rows(row, row2, selection))
93
94          self.table.rows = result
95          self.table.columns = columns
96          self.record = {v: i for i, v in enumerate(columns)}
97          self.variables = set(head.variables) if strict else set(self.variables.union(node.variables))
98
99
100     def compute(self, head: Head):
101         # Cas d'une racine sans enfants (ou requête à un seul atome)
102         if self.root and not self.children:
103             self.table.project(
104                 [v in head.variables for v in self.table.columns],
105                 lambda row: True
106             )
107             self.record = { v:i for i, v in enumerate(self.table.columns) }
108             self.variables = set(head.variables)
109             return
110
111         for index, child in enumerate(self.children):
112             child.compute(head)
113             self.full_join(child, head, self.root and index == len(self.children) - 1)
114
115
116 class Instance:
117     tables: dict[str, Table]  # Annuaire nom-table
118
119     def __init__(self):
120         self.tables = dict()
121
122     def add_table(self, table: Table):
123         self.tables[table.name] = table
124
125     # Crée l'arbre de jointure d'une requête
126     def join_dfs(self, tree, query: Query, seen, i):
127         seen.add(i)
128         jointure = JoinNode(query.atoms[i], self.tables[query.atoms[i].name])
129
130         for j in tree[i]:
131             if j not in seen:
132                 jointure.push_child(self.join_dfs(tree, query, seen, j))
133
134         return jointure
```

```python

    def jointure(self, query: Query):
        hyper = Hyper(len(query.variables))

        record = dict()
        #record_bis = dict()

        for i, v in enumerate(query.variables):
            record[v] = i
            #record_bis[i] = v

        for atom in query.atoms:
            arete = Arete()
            arete.augmenter(*[record[v] for v in atom.variables])
            hyper.ajouter_arete(arete)

        arbre = hyper.arbre_jointure()

        if not arbre:
            raise ValueError("La requête est cyclique (non supporté par l'algorithme de Yannakakis).")

        #arbre_bis = dict()

        #for i, adj in enumerate(arbre):
        #    arbre_bis[record_bis[i]] = [record_bis[j] for j in adj]

        vus = set()
        return self.join_dfs(arbre, query, vus, 0)

    # Passage bottom -> up (Remontée des contraintes vers la racine)
    def bottom_up(self, query: Query, node: JoinNode):
        for child in node.children:
            self.bottom_up(query, child)

            variables = node.variables.intersection(child.variables)
            if len(variables) > 0:
                node.semi_join(child) # Élimination des tuples du parent sans correspondance (bottom-up)

    # Passage top -> down (Propagation des contraintes vers les feuilles)
    def top_down(self, query: Query, node: JoinNode):
        for child in node.children:
            variables = node.variables.intersection(child.variables)

            if len(variables) > 0:
                child.semi_join(node) # Propagation de la réduction vers les feuilles (top-down)
            self.top_down(query, child)

    # Réalisation d'une requête
    def query(self, query: Query):
        # Vérification de la requête
        for atom in query.atoms:
            if atom.name not in self.tables:
                raise ValueError(f"Table '{atom.name}' non présente dans l'instance.")
            if len(atom.original) != len(self.tables[atom.name].columns):
                raise ValueError(
                    f"Nombre de colonnes incorrect pour l'atome {atom.name}"
                )

        # Création de l'arbre de jointure
        jointree = self.jointure(query)
        jointree.root = True

        # Premier passage
```

```
198         self.bottom_up(query, jointree)
199
200         # Deuxième passage
201         # Suit directement l'algorithme de Yannakakis, mais il est possible de sauter cette étape
202         self.top_down(query, jointree)
203
204         # On réalise la jointure
205         jointree.compute(query.head)
206
207         return jointree.table
```